

编程语言C++

欢迎来到计算机编程语言C++学习资料分享！此文档是个人在学习C++中积累的一些资料和个人总结，希望对看到的人有所帮助，同时也希望有知识、有能力的人一起完善这份文档

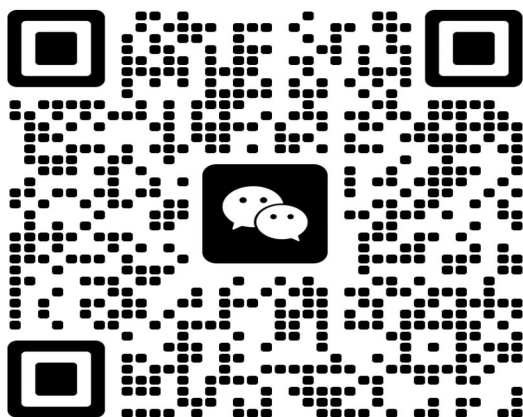
资料分享

面试八股总结



cpp辅导的阿甘

中国大陆 上海



扫一扫上面的二维码图案，加我为朋友。

本人微信感兴趣的可以加一下，有什么学习或者求职上的疑惑都可以私信我，尽量都会给大家解答的，并且周五晚上日常答疑聊天也会给大家解答



cpp辅导的阿甘 向你推荐

奔跑中的cpp / c++



微信扫码加入星球

这是自己创建的知识星球感兴趣的可以看看，星球介绍可以看下面的链接：

<https://mp.weixin.qq.com/s/INb6nA5ci7GhJMrUTSBSjw>

资料分享

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

书籍资料分享

1. C++ Primer Plus
2. effective c++
3. effective modern c++
4. 现代c++语言核心特性解析
5. STL源码剖析
6. c++并发编程

 Introduction · C++并发编程(中文版)

7. c++20

 C++20高级编程 (罗能) (Z-Library).pdf

8. c++提高编程

 C++提高编程PDF版.pdf

9. 侯捷11-14

【超级会员V1】通过百度网盘分享的文件：··侯捷 – C++标准11-14

链接：<https://pan.baidu.com/s/1prm6tf-ooMgOvsl476-nag>

提取码：b2wd

【超级会员V1】通过百度网盘分享的文件：侯捷视频

链接：<https://pan.baidu.com/s/1C1ETA7gaV9NXyEfH4y9STA>

提取码：b2wd

网站分享

1. 快速学习c++语法网站（菜鸟编程：有 基本语法教学和在线编辑器，无需自己再下载）

面试八股总结

希望看到的人，能够对他的学习、工作有帮助，同时也希望有能力的人可以一起来完善此文档，让它可以发挥出更大的价值。

封装

将具体的实现过程和数据封装成一个函数，通过接口进行访问，降低耦合性。

继承

概念

- 继承其实就是在保持原有类特性的基础上进行扩展，增加功能。这样产生的新类叫**派生类**，而被继承的类称基类。其实**继承主要就是类层次上的一个复用**。比如创建了一个动物的类，具有一些属性，但是呢具体到某类动物又有特有的属性，比如熊猫冬天爱睡觉，此时我们可以再创建一个熊猫的类，继承动物类，复用它的属性，并且也增加了自己所特有的属性。**(只回答这一部分就好)**
- 并且类的继承有三种方式：（可忽略）
 - public继承，那基类的public成员在派生类中依旧是public成员，基类的protected成员在派生类中依旧是protected成员，基类的private成员在派生类中不可见
 - protected继承，那基类的public成员在派生类中是protected成员，基类的protected成员在派生类中是protected成员，基类的private在派生类中不可见
 - private继承，那基类的public成员在派生类中是private成员，基类的protected成员在派生类中是private成员，基类的private成员在派生类中不可见

构造函数和析构函数的执行顺序

- 1.虚基类的构造函数（多个虚基类则按照继承的顺序执行构造函数）；
- 2.基类的构造函数（多个普通基类也按照继承的顺序执行构造函数）；
- 3.类类型的成员对象的构造函数（按照初始化顺序）；
- 4.派生类自己的构造函数

构造函数的执行顺序怎么样的（和析构函数反过来）

- 先调用所有基类的构造函数，再调用派生类中子对象的构造函数（如派生类中的子对象），最后调用派生类的构造函数。处于同一层次的各基类构造函数的调用顺序取决于派生类所指定的基类顺序，与派生类构造函数中所定义的成员初始化列表顺序无关。

为什么用成员初始化列表会快一些

初始化列表概念

在类的构造函数中，不在函数体内对成员变量赋值，而是在构造函数的花括号前面使用冒号和初始化列表赋值。

哪些初始化操作必须用到初始化列表

当初始化一个引用成员变量时；

初始化一个const成员变量时；

当调用一个基类的构造函数，而构造函数拥有一组参数时；

当调用一个成员类的构造函数，而他拥有一组参数；

编译器会一一操作初始化列表，以适当顺序在构造函数之内安插初始化操作，并且在任何显示用户代码前。list中的项目顺序是由类中的成员声明顺序决定的，不是初始化列表中的排列顺序决定的

效率（为什么快）

对于类型，它少了一次调用构造函数的过程，而在函数体中赋值则会多一次调用。而对于内置数据类型则没有差别。

由于对象成员变量的初始化动作发生在进入构造函数之前，对于内置类型没什么影响，但如果有些成员是类，那么在进入构造函数之前，会先调用一次默认构造函数，进入构造函数后所做的事其实是一次赋值操作(对象已存在)，所以如果是在构造函数体内进行赋值的话，等于是 一次默认构造加一次赋值，而初始化列表只进行一次赋值操作。

（注意：对于const成员变量，必须通过初始化列表进行初始化）

多重继承的问题

- 同一个对象的地址不同，比如一个子类C同时继承了基类A和B，创建了一个子类C的对象，然后分别用基类A和B的指针同时指向这个子类对象，但是得出的地址不一样；（通过多重继承得到的对象可能拥有不同的地址）
- **数据的冗余**，比如有一个Person类，然后学生类和老师类分别继承Person类，Doctor继承学生类和Student类，也会导致**二义性的存在**
 - 解决方法：虚继承
- 可能产生多个虚函数表
 - 需要进行强制类型转换的时候，推荐使用新式类型转换关键字dynamic_cast
- [C++中的多重继承_c++ 多重继承_SlowIsFastLemon的博客-CSDN博客](#)

菱形继承问题

- 概念：一个类C通过继承类A和类B，但是类A和类B又同时继承于公共基类N
- 带来的问题：带来二义性；类C中存在两份的基类N，分别存在类A和类B中，如果数据多则严重浪费空间
- 解决方法：引入虚基类，类A和类B虚继承基类N，在**间接继承共同基类时只保留一份基类成员**；对于基类的初始化是**由最后的派生类中负责初始化**；在最后的派生类中不仅要直接基类进行初始化，还要负责对虚基类初始化；C++编译系统**只执行最后的派生类对基类的构造函数调用**，而忽略其他派生类对虚基类的构造函数调用。从而避免对基类数据成员重复初始化。因此，虚基类只会构造一次
- [C++菱形继承问题和虚继承分析_Jimmy1224的博客-CSDN博客](#)

多态

概念

多态就是不同继承类的对象，对同一消息做出不同的响应，**基类的指针指向或绑定到派生类的对象**，使得基类指针呈现不同的表现方式。在基类函数的前加上virtual关键字，在派生类中

重写该函数，运行时将会根据对象的实际类型来调用相应的函数。如果对象类型是派生类，就调用派生类的函数；如果对象类型是基类，就调用基类的函数。（都答）

几种方式

虚函数virtual

重载

虚函数实现

- 如果一个类包含了虚函数，那么在创建该类的对象时就会额外地增加一个数组（保存在常量区），数组中的每一个元素都是虚函数的入口地址。并且还会在对象的开头位置设置一个指针，指向虚函数表，并且这个指针始终位于对象的开头位置。（虚函数表示是属于类的，而不属于具体的对象，一个类只需要一个虚函数表即可）
- 在虚函数表中，基类的虚函数在vtable中的索引（下标）是固定的。不会随着继承层次的增加而改变，派生类新增的虚函数放在数组的最后
- 不是虚函数，通过基类的指针只能访问派生类的成员变量，但是不能访问派生类的成员函数。而引入虚函数就是为了解决这个问题，让基类的指针能够访问派生类的成员函数。（此部分不需要作答）

基类的指针是怎么绑定派生类的

其实，如果类包含了虚函数，那么在该类的对象内存中还会额外增加类型的信息，即type_info对象。然后编辑器会在该虚函数表的开头插入一个指针，指向当前类对应的type_info对象。当程序在运行阶段获取类型信息时，可以通过对象指针p找到虚函数表指针vfptr，再通过vfptr指针找到type_info对象的指针，进行得到类型信息。（多态指针类型的转换）

为什么构造函数不能是虚函数

- 内存：在执行构造函数之前第一个对象尚未创建完成，虚函数表尚不存在，也没有指向虚函数表的指针，所以此时无法查询虚函数表，也就不知道调用哪一个构造函数
- 使用：构造函数是通过创建对象时自动调用的，不可能通过父类的指针或者引用去调用，所以规定构造函数不能是虚函数

为什么要把析构函数声明成虚函数

析构函数定义成虚函数是为了防止内存泄露，因为当基类的指针或者引用指向或绑定到派生类的对象时，如果未将基类的析构函数定义成虚函数，会调用基类的析构函数，那么只能将基类的成员所占的空间释放掉，派生类中特有的就会无法释放内存空间导致内存泄露。

为什么不能把虚函数声明成inline（内联）

inline是在编译器将函数类替换到函数调用处，是静态编译的。而虚函数是动态调用的，在编辑器并不知道需要调用的是父类还是子类的虚函数，所以不能够inline声明展开，所以编辑器会忽略

为什么（静态）static成员函数不能为virtual

- static成员不属于任何类对象或类实例，所以即使给此函数加上virtual也是没有任何意义的
- 静态成员函数是没有this指针的，而在虚函数中是通过this指针访问虚函数表指针的

纯虚类概念，以及为什么要有纯虚类（重点）

- 它没有实现体（即没有函数体），只有声明，被声明为纯虚函数的类被称为**抽象类**，不能直接实例化。抽象类中的纯虚函数需要在非抽象类中被实现，**否则非抽象类也是抽象类**。
- 如果在抽象类的派生类中没有重新说明纯虚函数，则该函数在派生类中仍然为纯虚类，而这个派生类仍然是一个抽象类（**前面必答**）
- 由于抽象类中至少包含有一个没有定义功能的纯虚函数，因此抽象类只能用作其他类的基类，**不能建立抽象类对象**
- 抽象类不能用作参数类型、函数返回类型或显示转换的类型。但可以声明指向抽象类的指针变量，此指针可以指向它的派生类，进而实现多态

构造函数和析构函数中调用虚函数

- 《Effective c++》的解释是派生类对象构造期间进入基类的构造函数时，对象类型变成了基类类型，而不是派生类类型。同样，进入基类析构函数时，对象也是基类类型

- 其实就是，从语法上讲，调用完全没有问题；但是从效果上看，往往不能达到需要的目的
- 比如，一个基类base构造函数和析构函数里调用Function虚函数，另一个继承这个基类的子类A的构造函数和析构函数里也调用这Function虚函数
- 然后在主程序中声明这个基类base指针，然后指向派生类A。这是会显示Base 类的构造函数中调用 Base 版本的虚函数，A 类的构造函数中调用 A 版本的虚函数；A 类的析构函数中调用 A 版本的虚函数，Base 类的析构函数中调用 Base 版本的虚函数
- 与预期不符，和普通函数一样

重载（overload）和重写（override）的区别

- 重写（覆盖）指的是在派生类中覆盖基类中的同名函数，**重写就是重写函数体，要求基类函数必须是虚函数**且：与基类的虚函数有相同的**参数个数**；与基类的虚函数有相同的**参数类型**；与基类的虚函数有相同的**返回值类型**
- 重写和重载的区别：1.重写是父类和子类之间的**垂直**关系，重载是不同函数之间的**水平**关系；2.重写要求**参数列表相同**，重载则要求参数列表不同，**返回值不要求**；3.重写关系中，调用方法根据**对象类型决定**，重载根据调用时实参表与形参表的**对应关系**来选择函数体（大概率都会问）
- 隐藏：隐藏指的是某些情况下，派生类中的函数**屏蔽**了基类中的同名函数，包括以下情况：1.两个函数参数相同，但是基类函数**不是虚函数**；2.两个函数参数不同，无论基类函数是不是虚函数，都会被隐藏。和重载的区别在于两个函数不在同一个类中；（此部分不问无需作答）

函数重载

- 原理
 - 底层的重命名机制将Add函数根据参数的个数，参数的类型，返回值的类型都做了重新命名。那么借助函数重载，一个函数就有多种命名机制
 - C++利用**name mangling(倾轧)技术**，来改变函数名，区分参数不同的同名函数
 - Name Mangling 是一种在编译过程中，将函数、变量的名称重新改编的机制。在C++重载、namespace等操作符下，函数可以有同样的名字，编译器为了区分各个不同地方的函数，将各个函数通过编译器内定的算法，将函数改成**唯一的名称**。
- 为什么C语言里面没有重载

- 在C语言中被解析为_Add, 三个一样, 所以不能进行区分, 因此C语言不支持函数重载

STL

答题要点

针对此部分在简历书写的时候, 建议直接书写**熟悉STL常用容器**即可。

写熟悉STL常用容器, 那面试官看见了肯定是直接问:

你都熟悉哪些常用容器啊?

回答这种比较大的问题, 还是那个套路。能多答就多答, 因为你答的肯定是你会的。如果你简单说下名字就不说了, 那人家面试官就自己问了, 假如问道一个自己不会的就挂了。并且面试时间是固定的, 你答的越多, 问其他问题时间就少。

答案, 就按照下面标注的答就好, 这些都答完也得10多分钟, 二十分钟了

array (静态固定的、和普通数组一样)

- array 容器的大小是固定的, 无法动态的扩展或收缩
- 使用 array 容器代替普通数组, 最直接的好处就是 array 模板类中已经为我们写好了很多实用的**方法**, 可以大大提高我们编码效率。 (**都答了**)

vector

- 介绍
 - vector是动态空间, 随着元素的加入, 它的内部机制会自动扩充空间以容纳新的元素 (**连续的内存空间**)
 - vector在增加元素时, 如果超过自身最大的容量, vector则将自身的容量扩充为**原来的两倍**。扩充空间需要经过的步骤: 重新配置空间, 元素移动, 释放旧的内存空间。一旦vector空间重新配置, 则指向原来vector的所有迭代器都失效了, 因为**vector的地址改变了**。
 - 并且数据存放在堆里 (答) 复制元素的方式是深拷贝的方式

- ----- (介绍只回答上面部分即可)
- 在不确定的情况下使用at而不是operator[]; 实际上at函数就是调用的operator[]函数, 只是多了一个检查是否越界的动作, 而operator[]函数是直接跳转位置访问元素, 所以速度是很快的, 从时间复杂度看, 是O(1)
- 注意的事项
- 迭代器的失效问题 (比如在某迭代器位置之前加一个值, 这个迭代器还可以使用吗)
 - 第一是在vector容器中间根据指定迭代器删除元素, 也就是调用erase函数, 此时因为当前位置会被后面的元素覆盖, 所以该指定迭代器会失效, 不过此时可以通过erase的返回值重新得到当前位置的正确迭代器;
 - 第二是在vector需重新申请内存的时候, 比如扩容, 比如释放未使用的内存等等这些过程中都会发生迭代器失效的问题, 因为内存有了变动, 此时就需要重新获得迭代器;
 - 有人说往vector中间插入数据也会使迭代器失效, 实际上根据源码是不会的, 看上面的insert实现可以得知, 再插入完成以后给当前迭代器重新赋值了的
- 如果用vector存一个数据量特别大的元素, 存的时候怎么存
- 内存释放怎么做
 - 常用的操作clear()和erase(), 实际上只是减少了size(), 清除了数据, 并不会减少capacity, 所以内存空间没有减少。那么如何释放内存空间呢, 正确的做法是swap()操作。
 - 通过swap函数, 用一个空的vector与当前vector进行交换 (vector().swap(v);), 这样v的内存空间等于被释放掉了, 而这个空vecctor因为是一个临时变量, 它在这行代码结束以后, 会自动调用vector的析构函数进行释放
 - 使用shrink_to_fit函数, 这个函数作用是释放掉未使用的内存, 先调用clear函数把所有元素清掉, 这样整块容器都变成未使用了, 然后再调用shrin_to_fit函数把未使用的部分内存释放掉
- emplace_back () 和push_back () 的区别
 - push_back () 向容器尾部添加元素时, 首先会创建这个元素, 然后再将这个元素拷贝或者移动到容器中 (如果是拷贝的话, 事后会自动销毁先前创建的这个元素)
 - emplace_back () 在实现时, 则是直接在容器尾部创建这个元素, 省去了拷贝或移动元素的过程

deque (双端队列)

■ 介绍

- deque 容器也擅长在序列尾部添加或删除元素（时间复杂度为 $O(1)$ ），而不擅长在序列中间添加或删除元素。deque 容器也可以根据需要修改自身的容量和大小。和 vector 不同的是，deque 还擅长在序列头部添加或删除元素，所耗费的时间复杂度也为常数阶 $O(1)$ 。并且更重要的一点是，deque 容器中存储元素并不能保证所有元素都存储到连续的内存空间中。（不必回答）

■ 底层实现

- deque 容器存储数据的空间是由一段一段等长的连续空间构成，各段空间之间并不一定是连续的，可以位于在内存的不同区域。为了管理这些连续空间，deque 容器用数组（数组名假设为 map）存储着各个连续空间的首地址。也就是说，map 数组中存储的都是指针，指向那些真正用来存储数据的各个连续空间
- 通过建立 map 数组，deque 容器申请的这些分段的连续空间就能实现“整体连续”的效果。换句话说，当 deque 容器需要在头部或尾部增加存储空间时，它会申请一段新的连续空间，同时在 map 数组的开头或结尾添加指向该空间的指针，由此该空间就串接到了 deque 容器的头部或尾部。
- -----答上面两部分
- 如果 map 数组满了怎么办？很简单，再申请一块更大的连续空间供 map 数组使用，将原有数据（很多指针）拷贝到新的 map 数组中，然后释放旧的空间。（此部分不问不需要作答）

■ 迭代器（此部分不问不需要作答）

- 构成
 - 迭代器内部包含四个指针：cur：指向当前正在遍历的元素；first：指向当前连续空间的首地址；last：指向当前连续空间的末尾地址；node：它是一个二级指针，用于指向map数组中存储的指向当前连续空间的指针

list

■ 介绍

- STL list 容器，又称双向链表容器，即该容器的底层是以双向链表的形式实现的。这意味着，list 容器中的元素可以分散存储在内存空间里，而不是必须存储在一整块连续的内存空间中

Stack

- **Stack（栈）底层一般用list或deque实现，封闭头部即可**，不用vector的原因是该容量大小有限制，扩容耗时（先进后出）
- **用队列实现栈的方法：**
- 使用两个队列实现栈：模拟入栈用一个队列1，模拟出栈的时候队列2用做备份，把队列1中除队列中的最后一个元素外的所有元素都备份到队列2中，然后弹出队列1的最后的元素，再把元素1从队列2导回队列1。（重点看下对应的算法题）
- 使用一个队列实现栈:模拟栈弹出元素的时候只要将队列头部的元素（除了最后一个元素）重新添加到队列尾部，此时再弹出元素的顺序就是栈的顺序

Queue

- **Queue（队列）底层一般用list或deque实现，封闭头部即可**，不用vector的原因应该是容量大小有限制，扩容耗时（stack和queue其实是适配器，而不是叫容器，因为是对容器的再封装）（先进先出）
- queue也可以使用list作为底层容器，不具有遍历功能，没有迭代器
- 用栈实现队列：用两个栈实现，一个是输入栈，另一个是输出栈。添加数据的时候，只要将数据放进输入栈即可。弹出数据的时候，如果输出栈为空，则把进栈数据全部导入输出栈，如果输出栈不为空，则直接从输出栈中弹出数据即可（重点看下对应的算法题）

set系列

- set
 - set底层实现
 - STL的set底层是用**红黑树**实现的，数值是有序的，数值不可重复，值不可修改，查询效率 $O(\log n)$ ，增删效率 $O(\log n)$
- multiset
 - multiset底层实现

- STL的map底层是用红黑树实现的，数值是有序的，数值可重复，值不可修改，查询效率 $O(\log n)$ ，增删效率 $O(\log n)$

■ unordered_set

- unordered_set底层实现
 - STL的unordered_set底层是用哈希表实现的，数值是无序的，数值不可重复，值不可修改，查询效率 $O(1)$ ，增删效率 $O(1)$

map系列

(此部分回答和set一样，只不过unordered_map建议多说)

■ map

- map底层实现
 - STL的map底层是用红黑树实现的，key是有序的，key的数值不可重复，key的值不可修改，查询效率 $O(\log n)$ ，增删效率 $O(\log n)$
- 使用 map 容器存储的各个键值对，键的值既不能重复也不能被修改。换句话说，map 容器中存储的各个键值对不仅键的值独一无二，键的类型也会用 const 修饰，这意味着只要键值对被存储到 map 容器中，其键的值将不能再做任何修改。
- operator[]和insert()效率对比
 - 当实现“向 map 容器中添加新键值对元素”的操作时，insert() 成员方法的执行效率更高；而在实现“更新 map 容器指定键值对的值”的操作时，operator[] 的效率更高

■ multimap

- multimap底层实现
 - STL的multimap底层是用红黑树实现的，key是有序的，key的数值可重复，key的值不可修改，查询效率 $O(\log n)$ ，增删效率 $O(\log n)$

■ unordered_map

- unordered_map的底层实现
 - STL的unordered_map底层是用哈希表实现的，key是无序的，key的数值不可重复，key的值不可修改，查询效率 $O(1)$ ，增删效率 $O(1)$
- 用“链地址法”（又称“开链法”）解决数据存储位置发生冲突的哈希表
- 当使用无序容器存储键值对时，会先申请一整块连续的存储空间，但此空间并不用来直接存储键值对，而是存储各个链表的头指针，各键值对真正的存储位置是各个链表

的节点。

- 将新键值对存储到无序容器中时，存储过程如下：
 - 将该键值对中键的值带入设计好的哈希函数，会得到一个哈希值（H）
 - 将H和无序容器拥有桶的数量n（链表的数量）做整除运算（即 $H \% n$ ），该结果即表示应将此键值对存储到的桶的编号；
 - 建立一个新节点存储此键值对，同时将该节点链接到相应编号的桶上。

迭代器失效

▪ vector

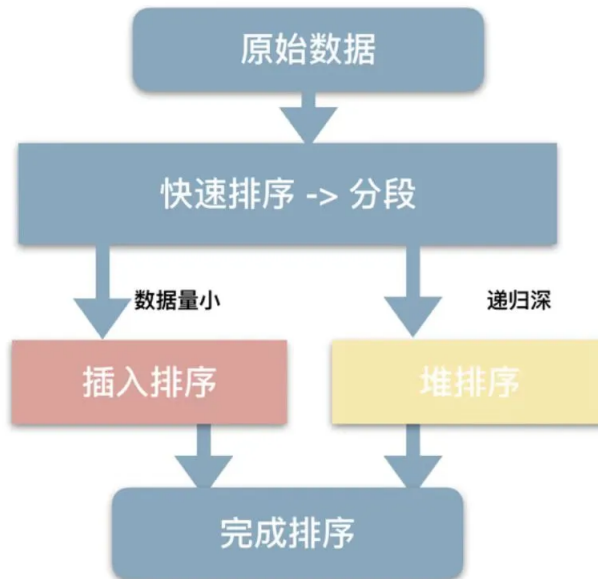
- 插入元素
 - 尾后插入：size < capacity时，首迭代器不失效尾迭代失效（未重新分配空间），size == capacity时，所有迭代器均失效（需要重新分配空间）。
 - 中间插入：中间插入：size < capacity时，首迭代器不失效但插入元素之后所有迭代器失效，size == capacity时，所有迭代器均失效。
- 删除元素
 - 尾后删除：只有尾迭代失效。
 - 中间删除：删除位置之后所有迭代失效。

▪ deque和vector的情况类似

- list双向链表每一个节点内存不连续，删除节点仅当前迭代器失效，erase返回下一个有效迭代器；
- map/set等关联容器底层是红黑树删除节点不会影响其他节点的迭代器，使用递增方法获取下一个迭代器 `mmp.erase(iter++)`；
- unordered_(hash) 迭代器意义不大，rehash之后，迭代器应该也是全部失效

算法

STL的sort算法，数据量大时采用**QuickSort快排算法**，分段归并排序⁹。一旦分段后的数据量小于某个门槛（16），为避免QuickSort快排的递归调用带来过大的额外负荷，就改用**Insertion Sort插入排序**。如果递归层次过深，还会改用**HeapSort堆排序**。



常用关键字的考查

static

- 静态成员变量
 - 不考虑类的情况：
 - 隐藏。所有不加static的全局变量和函数具有全局可见性，可以在其他文件中使用，**加了之后只能在该文件所在的编译模块中使用**
 - **默认初始化为0**，包括未初始化的全局静态变量与局部静态变量，**都存在全局未初始化区**
 - 静态成员变量在函数内定义，始终存在，且**只进行一次初始化**，具有记忆性，其作用范围与局部变量相同，**函数退出后仍然存在，但不能使用**。
 - 考虑类的情况：
 - 类的静态成员变量**只与类关联，不与类的对象关联**。定义时要分配空间，**不能在类声明中初始化，必须在类外定义外部初始化**，初始化时不需要标示为static；可以被非static成员函数任意访问
- 静态成员函数
 - `static int a () ;`

- 静态函数是否可以调用非静态成员函数
 - 在C++中，静态成员函数可以调用非静态成员函数，但需要通过对象或对象指针来调用。
- 非静态成员函数是否可以调用静态成员函数
 - 是的，C++中的非静态成员函数可以调用静态成员函数。
 - 非静态成员函数可以通过类名或类的对象来访问和调用静态成员函数。静态成员函数不依赖于类的对象，因此可以在非静态成员函数中直接使用类名或对象来调用静态成员函数。
- 静态成员函数是否可以调用类的构造函数
 - 在C++中，静态成员函数不能直接调用类的构造函数。
 - 静态成员函数是属于类的函数，它们不依赖于类的对象实例，因此没有隐式的this指针。由于构造函数用于创建类的对象，它是与对象实例相关联的，无法在静态成员函数中直接调用。
- static成员函数不能被virtual修饰，static成员不属于任何对象或实例，所以加上virtual没有任何实际意义；静态成员函数没有this指针，虚函数的实现是为每一个对象分配一个vptr指针，而vptr是通过this指针调用的，所以不能为virtual；虚函数的调用关系，this->vptr->ctable->virtual function。
- 普通成员变量存储在栈或堆中，而静态成员变量存储在静态全局区；
- 全局变量和static变量的区别
 - 非静态全局变量的作用域是整个源程序，当一个源程序由多个原文件组成时，非静态的全局变量在各个源文件中都是有效的。静态全局变量则限制了其作用域，即只在定义该变量的源文件内有效
 - static函数与普通函数最主要区别是static函数在内存中只有一份，普通函数在每个被调用中维持一份拷贝程序的局部变量存在于（堆栈）中，全局变量存在于（静态区）中，动态申请数据存在于（堆）

inline

- 内联函数在编译的时候直接将函数代码内嵌入到目标代码中，省去了函数的调用开销来提高效率，并且相比于宏定义它有参数类型检查，具有返回值，可以实现重载等函数有的功能，比宏定义更安全些（宏是在预处理阶段进行替换的）
- 还有就是不是将函数声明成内联函数不一定会是内联函数这只是个建议，最终还是要看编

辑器决定。

- 并且在类里面定义定义的函数默认是添加上inline的。但是不建议把构造函数和析构函数声明成内联函数，这是没有意义的。这是因为编辑器会在构造函数中添加额外的操作（申请/释放内存，构造/析构对象等），致使构造函数/析构函数并不像看上去那么简洁

define

宏定义只是在预处理阶段简单的进行 文本替换，不做类型检查和计算，也不求解，容易产生错误，一般最好加上一个大括号包含住全部的内容，要不然容易出错

struct 和class的区别 （重点）

- struct默认是公有的，class默认是私有的；class默认是private继承，struct默认是public继承
- 并且c++的struct和c的struct相比，还多了个支持函数定义的功能
- C语言和C++有什么区别

const

针对变量

普通变量：（1）常变量，值不可变；（2）定义const变量时，需要对它进行初始化；（3）如果类的成员变量是const类型的变量，那么该变量不能在类定义外部初始化，只能通过构造函数初始化列表进行初始化，并且必须有构造函数

指针：（1）分为顶层const和底层const；（2）*号的右边，const修饰的变量本身是一个常量；（3）*号的左边，const修饰的变量所指向的是一个常量

例子讲解：

针对函数：

类的成员函数，若指定为const类型，表明是常函数，不能修改类的成员变量，并且类的常对象只能访问类的常成员函数

volatile （嵌入式）

这个关键字声明的变量编译器对访问该变量的代码就不再进行优化；volatile定义变量的值是易变的，每次用到这个变量的值的时候都要去重新读取这个变量的值，而不是

读寄存器内的备份

mutable

- 可变的，易变的；在c++中，mutable也是为了突破const的限制而设置的；在const函数里面修改一些跟类状态无关的数据成员，那么这个变量就需要被mutable修饰
- mutable是用来修饰变量的，被修饰的变量，将永远处于可变的状态；1.在常函数里，被修饰的变量可以被修改；2.在常对象中也可以被修改

explicit

- explicit关键字用来修饰类的构造函数，被修饰的构造函数的类，不能发生相应的隐式类型转换，只能以显示的方式进行类型转换
- explicit用来修饰类的构造函数，只能用于类内部的构造函数声明上，用于单个参数的构造函数，且不能发生相应的隐式类型转换，只能以显示的方式进行类型转换

override

override的作用就出来了，它指定了子类的这个虚函数是重写的父类的，如果你名字不小心打错了的话，编译器是不会编译通过的。

final

当不希望某个类被继承，或不希望某个虚函数被重写，可以在类名和虚函数后添加final关键字，添加final关键字后被继承或重写，编译器会报错

strlen和sizeof

- sizeof是运算符，strlen是字符处理函数
- sizeof参数是任意的数据类型或者数据；strlen的参数只能是字符串指针且结尾是'\0'的字符串
- 如果参数是指针的话，sizeof计算的是指针的长度，而不是对应数据或者字符串的长度；

constexpr

- `constexpr`出现是为了解决`const`的双重语义：只读变量和修饰常量
- C++11标准为了解决`const`关键字的双重语义问题，保留了`const`表示“只读”的语义，而将“常量”的语义划分给了新添加的`constexpr`关键字
- 所以，C++11 标准中，建议将`const`和`constexpr`的功能区分开，表达“只读”语义的场景用`const`，表达“常量”语义的场景用`constexpr`
- 当然，`constexpr`修饰的函数也有一定的限制：（1）函数体尽量只包含一个`return`语句，多个可能会编译出错；（2）函数体可以包含其他语句，但是不能是运行期语句，只能是编译期语句；编译器会将`constexpr`函数视为内联函数！所以在编译时若能求出其值，则会把函数调用替换成结果值。
- `constexpr`还能修饰类的构造函数，即保证传递给该构造函数的所有参数都是`constexpr`，那么产生的对象的所有成员都是`constexpr`。该对象是`constexpr`对象了，可用于只使用`constexpr`的场合。注意`constexpr`构造函数的函数体必须为空，所有成员变量的初始化都放到初始化列表中。
- 函数返回值做变量，
- 浅谈 C++ 中的 `const` 和 `constexpr` – 知乎 (zhihu.com)

typeid

- 获取一个实例的类型
- 当`typeid`操作符的操作数是不带有虚函数的类类型时，`typeid`操作符会指出操作数的类型，而不是底层对象的类型
- 如果`typeid`操作符的操作数是至少包含一个虚拟函数的类类型时，并且该表达式是一个基类的引用，则`typeid`操作符指出底层对象的派生类类型
- C++中typeid的用法_c++ typeid用法_man_of_sillyy的博客-CSDN博客

deprecated

- 指示允许使用声明有此属性的名称或实体，但因故不鼓励使用。编译器通常会对其使用情况发出警告。若指定了 字符字面量，则它通常被包含于警告中。
- C++关键字deprecated_c++ deprecated_瞻邈的博客-CSDN博客

nullptr （重点）

- 在C语言中，使用NULL表示空指针，实际是一个宏，表示为0或者（void*）0。C语言中，是允许void指针隐式转换为其他类型指针的。但是在C++中，void指针是不可以隐式转换为其他类型指针的，必须显式转换。被定义为0
- 但是在c++的重载函数中会存在一个二义性的错误。比如有两个是函数名为fun的函数，一个函数的形参是int类型，一个函数的形参是char*。如果调用这个函数，传入的实参是个NULL，就会出现二义性
- nullptr是一个常量，通过模板可以隐式转换到所有的裸指针型别，但不具备整型型别

相关函数方法

默认函数有哪些

- 缺省构造函数、拷贝构造函数、析构函数、赋值运算符
- 默认构造、拷贝构造、拷贝复制运算符
- 析构函数
- 移动构造：移动构造函数用于通过移动已存在的对象来创建一个新对象，通常用于提高性能。如果类中包含指针成员变量或资源管理的成员变量，需要注意自定义移动构造函数以执行资源所有权的转移或资源管理的操作。className(className&& other)
- 移动赋值运算符：移动赋值运算符用于将一个对象的值移动给另一个已存在的对象，通常用于提高性能。className& operator=(className&& other)

拷贝构造函数

- 讲解
 - 在C++中，与类名同名，且形参是本来对象的引用类型的函数，叫做拷贝构造函数，

与构造函数一样，当我们不主动定义的时候，系统会自动生成一个，进行两个对象成员之间对应的简单赋值，用来初始化一个对象。

■ 为什么要有拷贝构造函数

- 一般的构造函数都是会完成一些成员属性初始化的工作，在对象传递给某一函数之前，对象的属性可能已经被改变了，如果在产生对象副本的时候再执行对象的构造函数，那么这个对象的属性又再恢复到原始状态，与预期不符。所以在产生对象副本的时候，构造函数不会被执行，被执行的是一个默认构造函数

■ 为什么拷贝构造函数必须声明为引用（重点）

- 如果拷贝构造函数中的参数不是一个引用，而是采用传值的方式，而传值的方式会调用该类的拷贝构造函数，从而造成无穷递归地调用拷贝构造函数，因此拷贝构造函数的参数必须是一个引用
- 不能声明成指针原因：如果把参数改成指针，是可以运行的。但是参数变指针后，已经不是拷贝构造函数了，已经变成了普通的构造函数，程序不会在需要拷贝构造函数的时候自动调用它

■ 调用时机：（在什么地方用）

- 用类的一个实例化对象去初始化另一个对象的时候
- 函数的参数是类的对象的时候（非引用传递）
- 函数的返回值是函数体内局部对象的类的对象时候

■ 深拷贝和浅拷贝（重点）

- 浅拷贝是只拷贝一个指针，并没有新开辟一个地址，拷贝的指针和原来的指针指向同一块地址，如果原来的指针所指向的资源释放了，那么再释放浅拷贝的指针的资源就会出现错误
- 深拷贝不仅拷贝值，还开辟出一块新的空间用来存放新的值，即使原来的对象被析构掉了，释放内存了也不会影响到深拷贝得到的值；在自己实现拷贝赋值的时候，如果有指针变量的话是需要自己实现深拷贝的

友元函数和友元类

- 友元函数和友元类是一种特殊的访问权限，它们允许函数或类访问其它类中的私有成员
- 友元函数：友元函数是在一个类中声明的非成员函数，它可以访问该类的私有成员和保护成员。通过将函数声明为类的友元函数，可以使函数在类的外部使用私有成员。友元函数

通常在类的声明中进行声明，并在类外部进行定义。

- 友元类：友元类是指一个类可以访问另一个类的私有成员和保护成员。通过将一个类声明为另一个类的友元类，可以使友元类能够访问该类的私有成员。友元类的声明通常放在类的声明中。
- 场景：访问私有数据成员；两个类之间共享私有信息

四种强制转换

reinterpret_cast

- 强制类型转换符用来处理无关类型转换的，主要用在：（1）任意指针之间的转换；（2）任意引用之间的转换；（3）指针和足够大的整型之间的转换
- `reinterpret_cast<Type>(expression)`
- `reinterpret_cast` 用于进行各种不同类型的指针之间、不同类型的引用之间以及指针和能容纳指针的整数类型之间的转换。转换时，执行的是逐个比特复制的操作。

const_cast

- `static_cast` 不能将 `const int*` 转成 `int *`，`const_cast` 就可以
- 应用：（1）常量指针被转化成非常量的指针，并且仍然指向原来的对象；（2）常量引用被转换成非常量的引用，并且仍然指向原来的对象；（3）`const_cast` 一般用于修改指针。如 `const char *p` 形式。
- `const_cast<Type>(expression)`
- `const_cast` 运算符仅用于进行去除 `const` 属性的转换，它也是四个强制类型转换运算符中唯一能够去除 `const` 属性的运算符。

static_cast

- `static_cast < type-id > (expression)`; 该运算符把 `expression` 转换为 `type-id` 类型，但没有运行时类型检查来保证转换的安全性
- 用于类层次结构中基类（父类）和派生类（子类）之间指针或引用的转换。注意：进行上行转换（把派生类的指针或引用转换成基类表示）是安全的；进行下行转换（把基类指针

或引用转换成派生类表示) 时, 由于没有动态类型检查, 所以是不安全的。

- 用于基本数据类型之间的转换, 如把int转换成char, 把int转换成enum。这种转换的安全性需要开发者来维护
- 把空指针转换成目标类型的空指针。
- 把任何类型的表达式转换成void类型。
- `static_cast`不能转换掉原有类型的`const`、`volatile`属性。
- 在c++ primer 中说道: c++ 的任何的隐式转换都是使用 `static_cast` 来实现。
- `static_cast` 不能用于在不同类型的指针之间互相转换, 也不能用于整型和指针之间的互相转换, 当然也不能用于不同类型的引用之间的转换
- move的原理说明下

dynamic_cast

- `dynamic_cast`专门用于将多态的基类的指针或引用强制转换为派生类的指针或引用, 而且能够检查转换的安全性。对于不安全的指针转换, 转换结果返回 `NULL` 指针。
- `dynamic_cast` 不能用于将非多态基类的指针或引用强制转换为派生类的指针或引用——这种转换没法保证安全性, 只好用 `reinterpret_cast` 来完成。

指针

○ 函数指针和指针函数的区别:

- 指针函数: 就是一个返回指针的函数, 其本质是一个函数, 而该函数的返回值是一个指针
- 函数指针: 其实就是一个指针变量, 该指针指向这个函数、总的来说, 函数指针就是指向函数的指针

○ 野指针和悬空指针:

- 野指针: 指的是没有被初始化过的指针。未初始化的指针的初始值可能是随机的, 使用未初始化的指针可能会导致段错误, 从而程序会崩溃。解决方法: 及时初始化, 要么置空
- 悬空指针: 指针指向一块内存空间, 当这块内存空间被释放后, 该指针依然指向这块内存空间。如果对悬空指针再次释放可能会出现不可预估的错误, 比如可能该段内存被别的程

序申请使用了，而此时对该段内存进行释放可能会产生不可预估的结果。解决方法：释放后立即置空

- this指针在哪里产生

- this指针其实是一个const指针，只能在成员函数内部使用。并且只有当对象被创建后this才有意义。
- this实际上就是成员函数的一个形参，在调用成员函数时将对象的地址作为实参传递给this。不过这个this形参是隐式的，并不出现在代码中，而是在编译阶段由编辑器默默地将它添加到参数列表中的。

- 指针和引用的区别（重点）

- 指针可以有多级，引用只有一级
- 指针可以为空，引用不能为NULL且在定义时必须初始化
- 指针在初始化后可以改变指向，而引用在初始化之后不可再改变
- sizeof指针得到的是本指针的大小，sizeof引用得到的是引用所指向变量的大小

C++11

智能指针

答题要点

同上述的STL中一样答题套路。

建议简历书写熟悉C++11的智能指针

面试官看见了肯定会直接问，你都熟悉哪些智能指针啊？

答的话，建议只答shared_ptr、weak_ptr即可。答的内容，下面有提示。

shared_ptr

- 采用引用计数器的方法，允许多个智能指针指向同一个对象，每当多一个指针指向该对象时，指向该对象的所有智能指针内部的引用计数加1，每当减少一个智能指针指向对象时，引用计数会减1，当计数为0的时候会自动的释放动态分配的资源
- 每一个由shared_ptr管理的对象都有一个控制块，除了包含一个引用计数，一个弱计数和其他一些数据
- -----(介绍shared_ptr把上面两部分说了，然后再回答说shared_ptr不是线程安全的，然后把面试题那个答案说了即可)

- 控制块规则：
 - **make_shared总是创建一个控制块。**
 - 从具备专属所有权的指针（即unique_ptr或auto_ptr指针）出发构造一个shared_ptr时，会创建一个控制块。
 - 当shared_ptr构造函数使用裸指针作为实参来调用时，会创建一个控制块。
 - 注意：从同一个裸指针出发来构造不止一个shared_ptr的话，会导致被指涉到的对象将会有多重的控制块。
- 具体原理
 - 智能指针将一个计数器与类指向的对象相关联，引用计数器跟踪共有多少个类对象共享同一指针
 - 每次创建类的新对象时，初始化指针并将引用计数置为1
 - 当对象作为另一对象的副本而创建时，拷贝构造函数拷贝指针并增加与之相应的引用计数
 - 对一个对象进行赋值时，赋值操作符减少左操作
 - 拷贝时，引用计数加1；赋值时，左值引用计数减1，右值引用计数加 1
- 性能影响
 - shared_ptr的尺寸是**裸指针的两倍**。因为它们内部即包含了一个指涉到该资源的裸指针，也包含一个指涉到该资源的引用计数的裸指针
 - **引用计数的内存必须动态分配。**
 - 引用计数的递增和递减必须是**原子操作**。存在并发读写的现象
- 注意事项
 - shared_ptr若是由std::make_ptr创建，可以避免动态分配的成本。
 - 不能使用一个原始地址初始化多个共享智能指针，会导致这个原始指针被初始化
- 面试问题
 - 多个shared_ptr指向同一个对象是否保证了线程安全
 - 不是；shared_ptr指针类有两个成员变量，一个是指向变量的指针，一个是资源被引用的次数。**引用次数加减操作内部自动加锁解锁，是线程安全的。**但是当指向变量的指针对象被多个线程访问它时，会出现线程安全的问题。当我们多个线程访问同一个shared_ptr时，应该要进行加锁操作。（这两步拷贝不会同时（原子）发生。）
 - shared_ptr的引用计数器是放在内存那个位置上
 - 在C++中，std::shared_ptr的引用计数器是存储在动态分配的控制块（control block）中的。控制块是一个特殊的数据结构，它包含了引用计

数、指向堆上分配对象的指针以及其他一些管理所需的信息。

- 通常情况下，控制块是通过动态内存分配（例如使用new运算符）从堆上分配的。std::shared_ptr的构造函数会为被管理的对象分配一个新的控制块，并将对象的指针保存在控制块中。每当创建一个新的std::shared_ptr来管理相同的对象时，这些std::shared_ptr会共享同一个控制块，其中的引用计数会递增。

weak_ptr

- 弱引用；（为了解决shared_ptr的问题）引用计数有一个问题就是互相引用形成环（环形引用），这样两个指针指向的内存都无法释放。需要使用weak_ptr打破环形引用。weak_ptr是一个弱引用，它是为了配合shared_ptr而引入的一种智能指针，它指向一个由shared_ptr管理的对象而不影响所指对象的生命周期，也就是说，它只引用，不计数。如果一块内存被shared_ptr和weak_ptr同时引用，当所有shared_ptr析构之后，不管还有没有weak_ptr引用该内存，内存也会被释放。所以weak_ptr不保证它指向的内存一定是有效的，在使用之前使用函数lock（）检查weak_ptr是否为空指针。
- 面试题
 - weak_ptr 的作用是什么
 - 解决环形引用问题，然后给它举例子

unique_ptr(不可也许，一般不会问，工作中一般用不到)

- 采用的是独享所有权语义，一个非空的unique_ptr总是拥有它所指向的资源。转移一个unique_ptr将会把所有权全部从源指针转移给目标指针，源指针被置空；所以，unique_ptr不支持普通的拷贝和赋值操作，不能用在STL标准容器中；如果拷贝一个unique_ptr，那么拷贝结束后，这两个unique_ptr都会指向相同的资源，造成在结束同时对同一内存指针多次释放而导致程序崩溃。
- 本质就是一个模板类，禁止拷贝构造和拷贝赋值（delete），实现移动构造和移动赋值
- unique_ptr使用move语义来管理动态分配的内存，即当一个unique_ptr对象被移动时，它的所有权也会被移动
- 明确显示了拷贝构造以及赋值操作是不能被编译通过的，如果想转移所有权，必须通过move函数来强制转换成右值，这样才能够进行移动资源所有权
- unique_ptr和裸指针有着相同的尺寸
- unique_ptr不允许复制，因为如果复制了一个std::unique_ptr,就会得到两个指涉到同一资源的std::unique_ptr，而这两者都认为自己拥有了该资源，应当析构

- `std::unique_ptr`以两种形式提供，一种是单个对象(`std::unique_ptr<T>`),另一种是数组(`std::unique_ptr<T[]>`)这样区分的结果是涉到的对象种类不会产生 二义性
- `std::unique_ptr`可以方便高效地转换成`std::shared_ptr`

auto_ptr(没必要说)

- 主要是为了解决“有异常抛出时发生内存泄露”的问题。因为发生异常而无法正常释放内存。

lambda表达式

- 建议看下，我之前写的博客，便瞬间秒懂了
 - [Lambda表达式从用到底层原理_lambda表达式底层-CSDN博客](#)
- 捕获方式
 - 空捕获，lambda不能使用所在函数中的变量
 - 值捕获，引用捕获
 - 值捕获 `[name] [=]`
 - 变量的值是 在lambda函数创建时拷贝，而不是调用时拷贝。由于被捕获的值是在lambda函数创建时拷贝，因此在随后对其修改不会影响到lambda内部的值
 - 引用捕获 `[&]`
 - 如果采用引用方式捕获变量，就必须保证被引用的对象在lambda执行的时候是存在的
 - 按引用的默认捕获模式可能导致空悬引用，一旦由lambda式所创建的闭包越过了局部变量或形参的生命周期，那么闭包内的引用就会空悬（即必须保证被引用的对象在lambda执行的时候是存在的
- 通过捕获列表，lambda函数可以访问父作用域中的非静态局部变量（静态局部变量可以直接访问，不能访问全局变量）
- 原理
 - 实际当我们编写了一个lambda表达式之后，编译器将该表达式翻译成一个未命名类的未命名对象。该类含有一个`operator()`。
 - 默认情况下，由lambda函数生成的类是`const`成员函数，所以变量的值不能修改。如果加上`mutable`，相当于去掉`const`。
- 面试题
 - C++函数可以返回一个lambda表达式对象吗

- C++的lambda表达式产生的对象确实可以作为函数的返回值。但是，如果你想返回一个lambda，那么需要注意类型问题。Lambda表达式的类型是在编译时自动生成的，每个Lambda表达式的类型都是唯一的，即使两个lambda表达式的形式完全相同。因此，你不能直接声明一个函数返回一个特定的lambda类型。解决这个问题的一种方法是使用std::function，它是一个通用的函数指针，可以容纳任何可以调用的对象，包括lambda。

-----下面这些东西了解下即可

在实际工作中，下面的就function用的多一些。

建议简历对这部分书写，写熟悉C++11的Lambda表达式、智能指针等

和面试官聊完上面的，可能还会问，其他的你还有熟悉的吗？

这个时候说下，move，完美转发，function即可。

右值

- 右值是指表达式结束后就不再存在的临时对象；有名字的对象是左值，右值没有名字（能不能对表达式取地址，如果能，则为左值，否则为右值）
- 分类：
 - 纯右值：非引用返回的临时变量；运算表达式产生的结果；字面常量；
 - 将亡值：与右值引用相关的表达式；例如：将要被移动的对象、T&&函数返回的值、std::move（）的返回值、转换成T&&的类型的转换函数的返回值。
- 右值引用作用：
 - 右值引用就是给右值取个名字；语法：类型数据&& 变量名=右值；（右值可以延长函数返回变量的生命周期）
 - 引入右值引用的主要目的是实现移动语义
 - 左值引用只能绑定（关联、指向）左值，右值引用只能绑定右值，如果绑定的不对，编译就会失败
 - 常量左值引用是一个万能的引用类型，它可以绑定非常量左值、常量左值、右值，只能读不能改。
- 左值引用与右值引用的区别

- 在C++中，左值引用（Lvalue Reference）和右值引用（Rvalue Reference）是引用类型的两种形式，它们有一些重要的区别。
- 1. 定义和语法：
 - 左值引用：使用&符号来定义，例如int&。
 - - 右值引用：使用`&&`符号来定义，例如`int&&`。
- 2. 绑定能力：
 - - 左值引用：只能绑定到左值（可寻址的对象），例如变量或具名对象。
 - - 右值引用：只能绑定到右值（不可寻址的临时对象），例如临时对象、字面量、表达式的结果。
- 3. 生命周期：
 - - 左值引用：一般用于引用长期存在的对象，生命周期可以与引用的对象相同。
 - - 右值引用：一般用于引用临时对象或将要销毁的对象，生命周期通常很短。
- 4. 修改能力：
 - - 左值引用：允许修改所引用的对象。
 - - 右值引用：通常用于移动语义和转移资源的所有权，允许修改所引用的对象。
- 5. 移动语义：
 - - 左值引用：不支持移动语义，只能通过拷贝构造函数进行值的传递。
 - - 右值引用：支持移动语义，可以通过移动构造函数或移动赋值运算符将资源从一个对象转移给另一个对象，避免不必要的拷贝，提高性能。
- 右值引用的引入主要是为了支持移动语义和完美转发（perfect forwarding），它们在C++11中被引入，并在后续的标准中得到了扩展和应用。
- 总结起来，左值引用和右值引用在绑定能力、生命周期、修改能力和移动语义等方面有明显的区别。左值引用更常用于引用长期存在的对象，而右值引用更适合处理临时对象和资源转移。

移动语义

- 原因：复制构造函数中，创建一些临时对象，而这些临时对象只是做数据的复制。通过移动语义可以直接将临时对象的内存移动到存储对象上，以避免内存数据的复制
- 用途：获取了被移动对象的资源（这里是内存）的所有权，同时把被移动对象的成员指针置为空（以避免移动过来的内存被析构），这个过程中没有新内存的申请和分配，在大量

对象的系统中，移动构造相对与拷贝构造可以显著提高性能！

- 实现：实现移动语义要增加两个函数，移动构造函数和移动赋值函数。
 - 移动构造函数的语法：类名（类名&& 源对象）{。。。。}
 - 移动赋值函数的语法：类名& operator=（类名&& 源对象）{。。。}
 - 移动构造实现步骤：（1）把资源从源对象中转移过来；（2）把源对象中的指针置空
- 注意事项：
 - 为了某些左值的局部变量，实现移动而不是拷贝，提供了move（）（一些对象的构造时可以获取到已有的资源（如内存）而不需要通过拷贝，申请新的内存，这样移动而非拷贝将会大幅度提升性能。例如有些右值即将消亡析构，这个时候我们用移动构造函数可以接管他们的资源。）（使用move意味着，把一个左值转换为右值，原先的值不应该再继续再使用（承诺即将废弃））方法来将左值转义为右值。用移动构造函数，左值对象被转移资源后，不会立刻析构，只有在离开自己的作用域的时候才会析构，如果继续使用左值中的资源，可能会发生意想不到的错误。
 - 如果没有提供移动构造/赋值函数，只提供了拷贝构造/赋值函数，编译器找不到移动构造/赋值函数就去寻找拷贝构造/赋值函数。
 - C++11中的所有容器都实现了移动语义，避免对含有资源的对象发生无谓的拷贝。
 - 移动语义对于拥有资源（如内存、文件句柄）的对象有效，如果是基本类型，使用移动语义没有意义。
- move原理：（移动语义主要由move实现）
 - 不管左值还是右值都转换成右值
 - 从实现上讲，move基本等同于一个类型转换：static_cast<T&&>(value)
 - 首先，通过右值引用传递模板实现，利用引用折叠原理将右值经过T&&（通用引用）传递类型保证不变还是右值，而左值经过T&&变为普通的左值引用，以保证模板可以任意实参，且保持类型不变。然后我们通过static_cast<>进行强制类型转换返回T&&右值引用，而static_cast之所以能使用类型转换，是通过remove_reference::type模板移除T&&，T&的引用，获取具体类型T（模板偏特化）。
 - remove_reference类其实就是利用模板的自动推导获取到了实参去引用后的类型
 - [std::move原理实现与用法总结_stdmove原理_~青萍之末~的博客-CSDN博客](#)
 - [C++高阶知识：深入分析移动构造函数及其原理 | 音视跳动科技 \(avdancedu.com\)](#)
- 引用折叠
 - 如果一个上下文中允许引用的引用存在（比如，模板函数的实例化），引用根据规则折叠为单个引用；如果任一引用为左值引用，则结果为左值引用。否则（即，如果引用都是右值引用），结果为右值引用（总结一句话就是左值引用总是折叠为左值引用，右值引用总是折叠为右值引用。）

- 发生的场景：（1）模板实例化；（2）auto变量的类型生成；（3）使用typedef和别名声明，在创建或者定义typedef过程中出现了引用的引用，则出现引用折叠就会起作用：typedef T&& RvalueRefToT;
- 模板类型推导
 - 模板参数类型是一个指针或引用，但不是通用引用：（1）当传递一个const对象给一个引用类型的参数时，传递的对象保留了常量性；（2）如果传递实参是一个引用，将忽略引用部分
 - 模板参数类型是一个通用引用：（1）如果传递的实参是左值，被推导为左值引用；（2）如果是右值，同一
 - 模板参数类型既不是指针也不是引用：传值给形参时会忽略常量性和易变性
 - 数组实参
 - 函数参数
- 场景：前提：1.定义的类使用了资源并定义了移动构造函数和移动赋值运算符；2.该变量即将不再使用；其实主要就是为了减小复制的开销

完美转发

- 概念：不仅能准确的转发参数的值，还能保证被转发参数的左右值属性不变
- 方案：
 - 如果模板中（包括类模板和函数模板）函数的参数书写成为T&&参数名，那么，函数即可以接受左值引用，又可以接受右值引用。
 - 提供了模板函数forward<T>(参数)，用于转发参数，如果参数是一个右值，转发之后仍然是右值引用；如果参数是一个左值，转发之后仍是左值引用
- 原理：
 - 聊聊C++中的完美转发 – 知乎 (zhihu.com)
 - forward实现了两个模板函数，一个接收左值，另一个接收右值
 - 然后forward模板函数对传入的参数进行强制类型转换(static_cast)，转换的目标类型符合引用折叠规则，因此左值参数最终转换后仍为左值，右值参数最终转成右值

function

- std::function就是C++中用来代替c函数指针的

- 类模版std::function是一种通用、多态的**函数封装**。std::function的实例可以对任何可以调用的目标实体进行存储、复制、和调用操作，这些目标实体包括普通函数、**Lambda**表达式、函数指针、以及其它函数对象等。std::function对象是对C++中现有的可调用实体的一种类型安全的包裹（我们知道像函数指针这类可调用实体，是类型不安全的）。它实质上是实现了一个泛型的回调机制。
- std::function 是一个**可调用对象包装器**，是一个**类模板**，可以容纳除了类成员函数指针之外的所有可调用对象，它可以用统一的方式处理函数、函数对象、函数指针，**并允许保存和延迟它们的执行**。
- std::function可以取代函数指针的作用，因为它可以**延迟函数的执行**，特别适合作为回调函数使用。它比普通函数指针更加的灵活和便利
- 返回类型(参数列表)

无锁操作

- 原子操作atomic
 - 原子操作std::atomic：类模板，保护一个变量
 - 步骤：（1）先read变量的值到CPU内存寄存器；（2）对寄存器的值递增；（3）将寄存器的值写回变量
 - cpu在执行任务的时候并不是直接从内存中加载数据，而是会先将数据加载到L1和L2 cache中（典型的是两层缓存，甚至可能更多），然后再从cache中读取数据进行运算。
 - 锁cache：在现代处理器上使用此种方式，但是在无法锁定cache的时候（如果锁驻留在不可缓存的内存中，或者如果锁超出了划分cache line 的cache boundy），仍然会去锁定总线。
 - [atomic的底层实现 – 王的博客 – 博客园 \(cnblogs.com\)](http://cnblogs.com)
 - 内存排序
 - 对于原子操作，一种重要的特性就是它们强制的内存排序（memory ordering）。内存排序是一个涉及到如何对内存操作进行排序的概念，它对编写并发代码非常重要。
 - std::memory_order_relaxed：放宽了对执行顺序的要求，**只保证了原子性，不保证内存顺序**。可以用于一些只需要原子操作而不需要严格内存顺序的场合。（自由顺序）。**在原子类型上的操作以自由序列执行，没有任何同步关系，仅对此操作要求原子性**。例如，在某一线程中，先写入A，再写入B。但是在多核处理器中观测到的顺序可能是先写入B，再写入A。自由内存顺序对于不同变量可以自由重

排序。这是因为不同的CPU缓存和内部缓冲区，在同样的存储空间中可以存储不同的值。对于非一致排序操作，线程没有必要去保证一致性。

- `std::memory_order_consume`：确保了当前线程看到的操作的顺序与它们在其他线程中的发生顺序一致。这可以用于建立"依赖关系"，比如一个线程写入一个指针和一个数据，另一个线程读取这个指针和数据。会对其标识的对象保证该对象存储先行于那些需要加载该对象的操作。
- `std::memory_order_acquire`：确保了某个特定的存储操作（或之前的操作）发生在当前操作之前。这可以用于实现锁。使用`memory_order_acquire`的原子操作，当前线程的读写操作都不能重排到此操作之前。例如，某一线程先读取A，再读取B，再以`memory_order_acquire`操作读取C，再读取D。在多核处理器中观测到的顺序D只能在C之后，不能出现先读取D，最后读取C的情况。但是，可能出现A或B重排到C之后的情况。
- `std::memory_order_release`：确保了某个特定的存储操作（或之后的操作）发生在当前操作之后。这也可以用于实现锁。使用`memory_order_release`的原子操作，当前线程的读写操作都不能重排到此操作之后。例如，某一线程先写入A，再写入B，再以`memory_order_release`操作写入C，再写入D。在多核处理器中观测到的顺序AB只能在C之前，不能出现C写入之后，A或B再写入的情况。但是，可能出现D重排到C之前的情况。
- `std::memory_order_acq_rel`：是`acquire`和`release`两种内存顺序的结合，用于读-修改-写操作。`memory_order_acq_rel`带此内存顺序的读-改-写操作既是获得加载又是释放操作。没有操作能够从此操作之后被重排到此操作之前，也没有操作能够从此操作之前被重排到此操作之后。
- `std::memory_order_seq_cst`：对于所有线程，对所有原子变量的操作都必须有一个单一的全局顺序。`memory_order_seq_cst`比`std::memory_order_acq_rel`更为严格。`memory_order_seq_cst`不仅是一个"获取释放"内存顺序，它还会对所有拥有此标签的内存操作建立一个单独全序。

模板

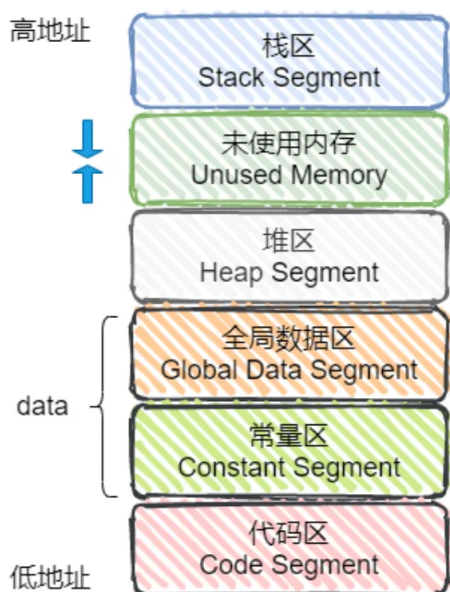
类模板中的静态变量，对于每一个具体类型，都会有不同的静态变量。比如对于`int`类型，其有多个对象，这些对象共享同一个静态变量。对于`char`类型，其有多个对象，这些对象也共享同一个静态变量，但是`int`类型对象共享的静态变量和`char`型共享的静态变量是不一样的。这是类模板的特性，因为`int`型对应的类和`char`型对应的类本身就不一样了。总之，在C++中，无论是对于函数模板还是类模板，c++编辑器都会进行两次编译，第一次编译是编译函数模板或则类模板本身的代码，第二次编译是编译具体类型替换后的代码，哪怕是

对于同一个模板，对于不同的类型，编译的代码都是不一样的，比如上述代码中的int和char,只不过由于模板的存在，而不用程序员两个类，c++编译器帮助我们做了。

内存相关

内存分布

C++中的内存分区，分别是堆、栈、全局/静态存储区、常量存储区和代码区。如下图所示



栈：在执行函数时，函数内局部变量的存储单元都可以在栈上创建，函数执行结束时这些存储单元自动被释放。栈内存分配运算内置于处理器的指令集中，效率很高，但是分配的内存容量有限

堆：就是那些由 new 分配的内存块，他们的释放编译器不去管，由我们的应用程序去控制，一般一个 new 就要对应一个 delete。如果程序员没有释放掉，那么在程序结束后，操作系统会自动回收

自由存储区：如果说堆是操作系统维护的一块内存，那么自由存储区就是C++中通过new和delete动态分配和释放对象的抽象概念。需要注意的是，自由存储区和堆比较像，但不等价。

全局/静态存储区：全局变量和静态变量被分配到同一块内存中，在以前的C语言中，全局变量和静态变量又分为初始化的和未初始化的，在C++里面没有这个区分了，它们共同占用同一块内存区，在该区定义的变量若没有初始化，则会被自动初始化，例如int型变量自动初始为0

常量存储区：这是一块比较特殊的存储区，这里面存放的是常量，不允许修改

代码区：存放函数体的二进制代码

内存泄露概念

内存泄露一般指的就是堆内存的泄露。堆内存是程序从堆中分配的，大小任意的内存块，使用完后必须显式释放内存。应用程序一般使用malloc、new等函数从堆中分配到内存块，使用完后程序必须负责相应的调用free或delete释放内存块，否则，这块内存就不能被再次使用，我们就说这块内存泄露了。

常见的内存泄露

- new和delete未成对出现
- 通过局部分配的内存，未在调用者函数体内释放
- 在局部分配的内存未释放
- delete void* 指针，void*指针指向一个类对象，delete的时候，导致没有调用到对象的析构函数，析构的所有清理工作都没有去执行从而导致内存的泄露。
- 没有将基类的析构函数定义为虚函数，当基类指针指向子类时，delete该对象时，不会调用子类的析构函数

内存泄露检测工具

- Linux下的是valgrind，该工具可以检测下列与内存相关的问题：（用valgrind工具查）
 - 未释放内存的使用
 - 对释放内存的读/写
 - 对已分配内存块尾部的读/写
 - 内存泄露
 - 不匹配的使用malloc/new/new[]和free/delete/delete[]
 - 重复释放内存
- 命令：valgrind --tool=memcheck --leak-check=full 执行程序（--tool=是指定valgrind使用的工具）通过这个命令看new/delete，malloc/free是否匹配
- 要使用Valgrind的Memcheck 工具来检测内存泄漏，可以按照以下步骤进行：
 - 编译源代码时，添加-g 选项启用调试信息，以便 Valgrind 可以提供更详细的报告。
 - g++ -g -o test test.cpp -pthread

- 运行 Valgrind 的 Memcheck 工具来分析程序并检查内存错误
 - `valgrind --tool=memcheck ./test`
- 面试官问，检测内存泄漏的方法，就说用 valgrind 工具查，然后把命令这一段说完即可
- 其实进入公司，针对内存泄漏检测，公司的系统团队会专门提供检测的方法

c++中怎么对内存泄露情况检查？ 怎么避免内存

- gcc 启用 asan 标志检查；重载 new 和 delete（可以用 hook）；使用 valgrind 工具
- 尽量使用智能指针代替普通指针；使用 RAII 的思想编写类；预先设定好内存池，使用内存池管理内存
- 标记清除法（参考其他语言 gc 的方法；公司有代码审查工具，可以自动检查野指针，空指针，内存泄漏等高风险问题，有问题会给出代码高风险预警

内存泄露造成的不利影响

一旦用户申请了内存分配而没及时释放，那么该部分内存存在整个程序运行周期内都是被占用的。因为内存未得到及时释放，从而可能导致可使用的动态内存空间会越来越少，一旦内存空间全部使用完成，则程序可能会导致因为内存不够而中止运行。

内存泄露检测组件的实现

- 聊到内存泄漏检测，建议主动说下，说自己实现了个内存泄漏检测组件，可以放到咱们公司项目代码里，运行看是否有内存泄漏的现象，然后说下面的方法
- 内存泄露检测具体实现的功能是能够检测出是否发生了内存泄露，并且能够判断代码代码哪一行引起了内存泄露
- 具体方法：
 - 使用 malloc/free 的底层 `_libc _malloc`, `_libc _free` 来实现 hook(具体过程)，使用 `_builtin _return _address` 函数来返回调用所在函数的代码段地址，进行定位内存泄露的位置；
 - malloc 的时候创建一个文件，文件名使用申请内存的地址，并记录申请该内存的代码段地址，free 的时候删除对应的文件，最终看是否有文件的存在即可判断是否存在内存泄露（回答到这就行）

- 并且注意自己实现的malloc的时候，如果执行printf会导致malloc一直在里面循环，这是因为printf本身内部就实现了一个malloc，这样会陷入死循环，为此设计了标识位，来判断是否使用自己的malloc，在printf前将其置为0，printf后将其置为1，来避免这个问题。

hook操作

■ 技术介绍

- hook技术又叫作钩子技术，它就是在程序运行的过程中，对其中的某个方法进行重写；在原有的方法前后加入我们自定义的代码。相当于在系统没有调用该函数之前，钩子程序就先捕获该消息，可以先得到控制权，这时钩子函数便可以加工处理（改变）该函数的执行行为。

■ 基于修改sys_call_table的系统调用挂钩

- Linux内核中所有的系统调用都是放在一个叫做sys_call_table的数组中，数组的值就表示这个系统调用服务程序的入口地址
- 将这个数组中存储的系统调用的地址改成我们自己的程序地址，就可以实现系统调用劫持（但是这个sys_call_table的内存页是只读属性）

• 步骤

- 获得sys_call_table的内存地址

- `grep sys_call_table /boot/ -r`

- 关闭写保护

- 控制页表只读属性是由CR0寄存器的WP位控制的，只要将这个位清零就可以对只读页表进行修改
- 查找虚拟地址所在页表地址（lookup_address）
- 设置只读页表属性

- 修改sys_call_table

- 找到要修改的系统调用的调用号（sys_call_table数组的下标）（`cat /usr/include/asm/unistd_64.h`）
- 指针替换就好

- 向linux内核中添加编译的内核模块（Linux内核是Linux 5.3.0-40）

- Linux内核开发所必需的头文件（3个init、module、kernel）
- 模块的许可证，module_license（查看完整列表）
- 编写init（加载）和exit（卸载），定义为静态的
- 调用 module_init 和 module_exit 函数告诉内核哪些函数是内核模块的加载

和卸载函数

- 编写makefile, make
- 使用insmd将编译的模块加载进内核进行测试

▪ 动态链接库的方法:

- 主要就是通过动态库的**全局符号介入功能**, 用自定义的接口来替换掉同名的系统调用接口。由于系统调用接口基本上是由C标准函数库libc提供的, 所以这里要做的事情就是用自定义的动态库来覆盖掉libc中的同名符号
- 外挂式
 - 通过优先加载自定义动态库来实现对后加载的动态库进行hook, 这种hook方式不需要重新编译代码
 - gcc编译生成可执行文件时会默认链接libc库, 所以不需要显式指定链接参数 (使用ldd命令查看可执行程序依赖的共享库)
 - 步骤:
 - 在不重新编译代码的情况下, 用自定义的动态库来替换可执行程序的系统调用
 - 建立一个c文件, 里面设计的函数签名和libc提供的库函数一样 (参数返回值之类的), 可以用syscall的方式调用对应编号的系统调用, 然后再添加几种内容
 - 将这个.c文件编译成动态库 (gcc -fPIC -shared hook.c -o libhook.so)
 - 通过设置 **LD_PRELOAD** 环境变量, 将libhook.so设置成**优先加载**, 从而覆盖掉libc中的库函数 (# LD_PRELOAD="./libhook.so" ./a.out)
 - LD_PRELOAD环境变量, 在指明在可执行程序运行之前, **系统会优先把咱们自定义的动态库加载到程序的进程空间**, 使得在可执行程序之前, 其**全局符号表中就已经有了一个系统调用符号**, 这样在后序加载libc共享库时, 由于全局符号介入机制, libc中的write符号不会再被加入全局符号表, 所以全局符号表中的系统调用就变成 我们自己实现
- 侵入式
 - 需要改造代码或是重新编译一次以指定动态库加载顺序
 - 把改造的系统调用直接放在可执行程序中
 - 编译时, 全局符号表里先出现的必然是可执行程序中的
 - 如果不改造代码, 那么可以重新编译一次, 通过编译参数将自定义的动态库放在libc之前进行链接 (gcc main.c -L. -lhook -Wl,-rpath=.) (由于默认情况下gcc总会链接一次libc, 并且libc的位置也总在命令行所有参数后面, 所以只需要像下面这样操作就可以了)

- 对于全局符号介入机制覆盖的系统调用接口，如何找回的方法
 - 因为程序运行时，依赖的动态库无论是先加载还是后加载，最终都会被加载到程序的进程空间中，也就是说，那些因为加载顺序靠后而被覆盖的符号，它们只是被“雪藏”了而已，实际还是存在于程序的进程空间中的，通过一定的办法，可以把它们再找回来
 - 可以使用Linux中的dslym方法找回被覆盖的符合

new和malloc的实现原理和区别

■ malloc实现原理：

- 这个函数底层是由brk、mmap、munmap系统调用实现的
- brk将数据段（.data）的最高地址指针_edata往高地址推，mmap是在进程的虚拟地址空间找一块空闲的虚拟内存
- malloc分配小于128k的内存，使用brk分配内存，将_edata往高地址推；malloc大于128k的内存，使用mmap分配内存；brk分配的内存需要等到高地址内存释放以后才能释放，而mmap分配的内存可以单独释放
- malloc从堆里申请内存，操作系统中有一个记录空闲内存地址的链表。当操作系统收到程序的申请时，就会遍历该链表，然后就寻找第一个空间大于所申请空间的堆结点，然后将该结点从空闲结点链表中删除，并将该结点的空间分配给程序。

■ new实现原理

- 内置类型直接调用operator new分配内存；对于复杂结构，先调用operator new分配内存，然后在分配的内存上调用构造函数
- 过程：（1）new表达式调用一个名operator new()函数，分配一块足够大小的、原始的、未命名的内存空间；（2）编译器运行相应的构造函数以构造这些对象，并为其传入初始值；（3）对象被分配了空间并构造完成，返回一个指向该对象的指针
- delete内置数据类型默认只是调用free函数，复杂数据类型先调用析构函数再调用operator delete；

■ [C++中placement new操作符（经典）_zhangxinrun_业余erlang的博客-CSDN博客](#)

■ 区别

- malloc和free是标准库函数，支持覆盖；new和delete是运算符，支持重载。
- malloc和free返回的是void类型指针（必须进行类型转换），new和delete返回的是具体类型指针。

- malloc仅仅分配内存空间，free仅仅回收空间，不具备调用构造函数和析构函数功能，用malloc分配空间存储类的对象存在风险；new和delete除了分配回收功能外，还会调用构造函数和析构函数。
- new 开辟的内存可以 free 释放掉吗
 - 先说一下各自执行过程
 - 如果通过new分配的内存，再用free去释放，就会少一步调用析构函数的过程。对于类对象而言，如果构造函数里面有申请的内存因为没有调用析构函数，所以该内存并没有释放

段错误 (coredump)

- 段错误有哪些
 - 概念：段错误是指访问的内存超出了系统所给这个程序的内存空间，其实就是访问了不可访问的内存，这个内存要么是不存在的，要么是受到系统保护的
 - 使用非法的内存地址（指针），包括使用未经初始化及已经释放的指针、不存在的地址、受系统保护的地址、只读的地址等（往受到系统保护的内存地址写数据，比如试图往内存地址0处写东西。char *p; p=NULL; *p='x';)
 - 内存读写越界。包括数组访问越界，或在使用一些内存的函数时，长度指定不正确或者这些函数本身不能指定长度（数组越界，变量类型不一致等），典型的函数有sprintf (snprint) 等等。
 - 函数不要返回其中局部对象的引用或地址，当函数返回时，函数弹出，局部对象的地址将失效，改写或读这些地址都会造成未知的后果。
 - 指针越界
 - 某个函数内开的数组过大，导致该函数的栈无法容纳数组，造成爆栈
- 排查段错误的方法有哪些
 - 通过core文件以及gdb调试查找
 - 当程序崩溃时，内核可能把该程序当前内存 映射到core文件里，如果没有可以用ulimit -c unlimited生成
 - 再通过gdb来调试core文件

概念

Linux 平台下最常用的一款程序调试器。

功能：

1. 程序启动时，可以按照我们自定义的要求运行程序，例如设置参数和环境变量；
2. 可使被调试程序在指定代码处暂停运行，并查看当前程序的运行状态（例如当前变量的值，函数的执行结果等），即支持断点调试；
3. 程序执行过程中，可以改变某个变量的值，还可以改变代码的执行顺序，从而尝试修改程序中出现的逻辑错误。

用法

调试命令 (缩写)	作用
(gdb) break (b)	在源代码指定的某一行设置断点，其中xxx用于指定具体打断点位置
(gdb) run (r)	执行被调试的程序，其会自动在第一个断点处暂停执行。
(gdb) continue (c)	当程序在某一断点处停止后，用该指令可以继续执行，直至遇到断点或者程序结束。
(gdb) next (n)	令程序一行代码一行代码的执行。
(gdb) step (s)	如果有调用函数，进入调用的函数内部；否则，和 next 命令的功能一样。
(gdb) until (u) (gdb) until (u) location	当你厌倦了在一个循环体内单步跟踪时，单纯使用 until 命令，可以运行程序直到退出循环体。 until n 命令中，n 为某一行代码的行号，该命令会使程序运行至第 n 行代码处停止。
(gdb) print (p)	打印指定变量的值，其中 xxx 指的就是某一变量名。
(gdb) list (l)	显示源程序代码的内容，包括各行代码所在的行号。
(gdb) finish (fi)	结束当前正在执行的函数，并在跳出函数后暂停程序的执行。
(gdb) return (return)	结束当前调用函数并返回指定值，到上一层函数调用处停止程序执行。
(gdb) jump (j)	使程序从当前要执行的代码处，直接跳转到指定位置处继续执行后续的代码。
(gdb) quit (q)	终止调试。

```

gdb调试工具： 大前提：程序是你自己写的。

-g：使用该参数编译可以执行文件，得到调试表。

gdb a.out

list: list 1 列出源码。根据源码指定 行号设置断点。

b: b 20 在20行位置设置断点。

run/r: 运行程序

n/next: 下一条指令（会越过函数） I

s/step: 下一条指令（会进入函数）

p/print: p i 查看变量的值。

continue: 继续执行断点后续指令。

quit: 退出gdb当前调试。

```

动态库/静态库

学习资料（可知道概念和优缺点以及区别）

https://blog.csdn.net/weixin_51483516/article/details/120837316

制作

动态库制作及使用：

1. 将 .c 生成 .o 文件，（生成与位置无关的代码 -fPIC）

```
gcc -c add.c -o add.o -fPIC
```
2. 使用 gcc -shared 制作动态库

```
gcc -shared lib库名.so add.o sub.o div.o
```
3. 编译可执行程序时，指定所使用的动态库。 -l: 指定库名 -L: 指定库路径。

```
gcc test.c -o a.out -lmymath -L./lib
```
4. 运行可以执行程序 ./a.out

静态库制作及使用步骤：

1. 将 .c 生成 .o 文件

```
gcc -c add.c -o add.o
```
2. 使用 ar 工具制作静态库

```
ar rcs lib库名.a add.o sub.o div.o
```
3. 编译静态库到可执行文件中：

```
gcc test.c lib库名.a -o a.out
```

额外扩展

C++编译的基本步骤（重点）



CSDN @hyppc9709

预处理：把一些`#define`的宏定义完成文本替换，然后将`#include`的文件里的内容复制到`.cpp`文件里，如果`.h`文件里还有`.h`文件，就递归展开。在预处理这一步，代码注释直接被忽略，不会进入到后续的处理中，所以注释在程序中不会执行。

编译：只是把我们写的代码转为汇编代码，它的工作是检查词法和语法规则，所以，如果程序没有词法或则语法错误，那么不管逻辑是怎样错误的，都不会报错。

汇编：将上一步的汇编代码(`main.s`)转换成机器码(machine code)，这一步产生的文件叫做目标文件(`main.o`)，是二进制格式。

链接：C/C++代码经过汇编之后生成的目标文件(`*.o`)并不是最终的可执行二进制文件，而仍是一种中间文件(或称临时文件)，目标文件仍然需要经过链接(Link)才能变成可执行文件。

<https://zhuanlan.zhihu.com/p/618037867>

简历书写

针对总结的这部分资料来说，简历书写建议如下内容：

技能特长

- 熟悉 c++ 的封装、继承、多态，STL 常用容器，熟悉 C++11 的 Lambda 表达式、智能指针等

根据辅导的这些学员经验，面试照着上面背读基本都可过，覆盖率99%